
Blackcoin

Quantum Quasar

Protocol V4

A Post-Quantum, Participation-First Evolution of Blackcoin

Technical White Paper

Quantum Quasar Developers

Version 30.1.1 Candidate - July 2026

<https://github.com/Blackcoin-Dev/Blackcoin>

Contents

Abstract	4
1. Design Philosophy: Participation Over Passivity	4
2. The V4 Timeline: Four Phases	5
Exact schedule (mainnet)	5
3. Post-Quantum Cryptography in Blackcoin	6
3.1 Why elliptic curves are the problem	6
3.2 ML-DSA-44: the quantum-safe signing path	6
3.3 New witness versions and address types	6
3.4 Self-test on startup	6
4. The Gold Rush Reward Epoch	7
4.1 The whitelist snapshot	7
4.2 The reward schedule	7
4.3 The PoS / PoW split	8
4.4 Qualifying and claiming: QQ SIGNAL and QQSPROOF	8
5. Quantum Migration and the Legacy Lockout	8
5.1 The problem being solved	8
5.2 The migration path	9
5.3 The lockout, and why it is a feature	9
6. Demurrage: Liveness as a Public Good	9
6.1 Exactly which coins are subject	10
6.2 The inactivity clock and the grace period	10
6.3 The decay curve	10
6.4 Staying at 100% is automatic and nearly free	11
7. Quantum Staking: Tiered, Cold, and Pooled	11
7.1 Tiered self-staking	12
7.2 Cold staking: separate the owner key from the staking key	12
7.3 Operator bonds and the per-pool cap	12
7.4 Autonomous redelegation	12
8. Reserved v15 EUTXO Design and RGB	13
8.1 EUTXO witness v15 is frozen in v30.1.1	13
8.2 RGB, client-side fixed-supply assets	13

9. Full Wallet and RPC Reference	13
9.1 Chain and schedule	13
9.2 Staking, Gold Rush, and mining	14
9.3 Quantum addresses and migration	14
9.4 Quantum staking, cold staking, operator bonds	14
9.5 Demurrage	15
9.6 EUTXO inspection and RGB	15
9.7 Wallet maintenance	15
9.8 The GUI	15
10. Worked Examples and Community Playbook	16
Example A, The long-term HODLer (recommended path)	16
Example B, The active staker during Gold Rush	16
Example C, The forgotten wallet	16
Example D, Running a staking pool	17
11. Economic Analysis: Why This Increases Participation	17
12. Security Considerations	18
Appendix A: Consensus Constant Reference	18
Appendix B: Glossary	20

Abstract

Blackcoin was one of the first pure Proof-of-Stake (PoS) cryptocurrencies, launched in 2014. Proof-of-Stake secures a network with coins rather than energy, which makes the long-term integrity of the chain depend on two things: the secrecy of the signing keys that protect stake, and the willingness of coin holders to actually *use* those keys to mint blocks. Both of these are now under pressure. The arrival of cryptographically relevant quantum computers threatens every coin that is protected only by elliptic-curve (ECDSA/Schnorr) signatures, and the long tail of dormant, never-staked coins slowly erodes the active security budget of any PoS chain.

Quantum Quasar (Protocol V4) is Blackcoin's answer to both problems at once. It introduces a NIST-standardized post-quantum signature scheme (ML-DSA-44) as a first-class, consensus-enforced spending path; a time-boxed, deterministic migration from legacy elliptic-curve outputs to quantum-safe outputs; and a set of participation mechanics, the Gold Rush reward epoch, liveness demurrage, tiered and cold quantum staking, and a bounded legacy lockout, that are explicitly designed to convert passive holding into active network security **without punishing the holder who is willing to participate**.

This paper documents the V4 protocol in exhaustive detail: every consensus constant, every phase boundary, every reward formula, and the exact wallet workflows and RPC commands a user needs. All numbers in this document are taken directly from the v30.1.1 candidate source and are annotated with the file that defines them.

1. Design Philosophy: Participation Over Passivity

Every design decision in Quantum Quasar answers a single question: *does this reward the people who actively secure the network, and does it give everyone else a clear, fair, fully-rewarded path to join them?*

Traditional PoS has a free-rider problem. A large holder can leave coins in a cold wallet for years, contribute nothing to block production, and yet retain the full economic weight of those coins. Meanwhile the holders who run nodes and mint blocks carry the real security burden. Over time this concentrates security into fewer hands and leaves the chain vulnerable, because so much of the coin supply is "asleep."

Quantum Quasar is built on the opposite premise: **the network should reward participation and slowly reclaim the influence of pure inactivity, while making participation cheap, automatic, and profitable for anyone willing to do it**. The three pillars below all serve that goal.

- **Full rewards for HODLers who help secure.** The Gold Rush epoch pays a large, deterministic bonus emission to holders, but only through the act of staking (PoS) or mining (PoW). Holding qualifies you; participating pays you. A holder who simply keeps their node online and staking earns their full share.
- **Liveness demurrage instead of dead weight.** After the migration era, quantum holdings that remain inactive for more than six months begin a slow, capped decay. Decayed principal is burned when spent; it is never added to transaction fees or paid to a miner or staker. The point is to ensure that keys are alive and that the security weight of the supply reflects who is actually present. A single automatic, near-free liveness attestation every few months (the wallet does it for you) keeps a direct quantum holding current. A cold-stake output is also subject to the activity clock; a successful coin stake spends and recreates it, resetting that clock.
- **A bounded, well-signposted quantum migration.** Legacy elliptic-curve outputs are the network's quantum attack surface. Gold Rush keeps ordinary quantum funding disabled so the base chain remains legacy-compatible. It is followed by an **18-month Migration phase** in which every holder can move coins into quantum-safe addresses with a one-click wallet operation. Final Lockout then closes the legacy spending path. Legacy value stays spendable for the full Gold Rush-plus-Migration schedule, but the migration transaction itself must be made during Migration.

The result is a network that trends toward *more* active nodes, *more* distributed security, and *more* engaged holders over time, the opposite of the slow ossification that afflicts passive-holding chains.

2. The V4 Timeline: Four Phases

Protocol V4 uses a complete, height-authoritative lifecycle on Blackcoin mainnet. MTP anchors remain as nominal forecasts and for isolated compatibility tests, but timestamp movement cannot advance, delay, skip, or reverse a mainnet phase boundary.

Phase	Mainnet heights	Target duration	Legacy spend?	v14/v16 funding and spending?	Gold Rush rewards?
Legacy	through 5,949,999	-	Yes	No	No
Gold Rush	5,950,000-6,192,999	243,000 blocks (~180 days)	Yes	No; shadow credits are locked	Yes
Migration	6,193,000-6,921,999	729,000 blocks (~540 days)	Yes	Yes	No
Final Lockout	from 6,922,000	permanent	No	Yes	No

The quantum column refers only to authenticated witness-v14 and witness-v16 paths. Witness v15 has no supported funding or spending workflow in any v30.1.1 phase; consensus rejects v15 outputs and spends from Migration onward.

Exact schedule (mainnet)

The consensus boundaries are defined in `src/shadow_schedule.cpp`, `src/shadow.h`, `src/consensus/params.h`, and `src/kernel/chainparams.cpp`.

- **V4 and Gold Rush begin:** height **5,950,000**.
- **Last Gold Rush block:** height **6,192,999**. Migration begins at **6,193,000**.
- **Last Migration block:** height **6,921,999**. Final Lockout and automatic demurrage begin together at **6,922,000**.

The retained time anchors are non-authoritative mainnet forecasts:

- `QUANTUM_QUASAR_MAINNET_V4_TIME = 1783835299` is **2026-07-12 05:48:19 UTC**.
- Adding the 180-day nominal Gold Rush duration gives `1799387299`, **2027-01-08 05:48:19 UTC**.
- Adding the 540-day nominal Migration duration gives `1846043299`, **2028-07-01 05:48:19 UTC**.

Phase membership is computed by `GetQuantumLifecycleState(nTime, nHeight)` (`src/consensus/params.h`). Because Blackcoin mainnet targets a **64-second block time**:

- **1,350 blocks/day, 40,500 blocks/month** (30-day month), **~493,000 blocks/year**.

Legacy elliptic-curve coins remain spendable for Gold Rush and Migration, approximately 720 target days in total. Ordinary quantum funding is deliberately off during the first 180 target days, so holders perform `migratetoquantum` during the following 540-day Migration phase. Final Lockout closes the legacy path at height 6,922,000.

3. Post-Quantum Cryptography in Blackcoin

3.1 Why elliptic curves are the problem

Blackcoin's legacy outputs are protected by ECDSA over secp256k1. A sufficiently large quantum computer running Shor's algorithm can recover a private key from a public key. Any coin whose public key is exposed on-chain, which includes every P2PK coinstate output Blackcoin has ever produced, and any address that has spent before, is at risk the moment such a machine exists. PoS chains are especially exposed because staking continuously publishes public keys.

3.2 ML-DSA-44: the quantum-safe signing path

V4 introduces **ML-DSA-44** (Module-Lattice Digital Signature Algorithm, the NIST FIPS 204 standardization of CRYSTALS-Dilithium at security level 2) as a native, consensus-verified signature scheme, provided by liboqs 0.15.0 and wrapped in `src/crypto/mldsa.h`. ML-DSA's security rests on the hardness of module lattice problems, which are not known to be broken by quantum algorithms.

Quantity	Size
Public key	1,312 bytes
Secret key	2,560 bytes
Signature	2,420 bytes

These are large compared to a 33-byte ECDSA public key and ~72-byte signature, the price of quantum resistance, which is why V4 places quantum data in the witness (where it is discounted) and uses commitment-based addresses so that the large key is only revealed at spend time.

3.3 New witness versions and address types

V4 defines three new SegWit witness versions, each a 32-byte commitment (`src/consensus/quantum_witness.h`, `src/addresstype.h`). All use the mainnet Bech32 human-readable prefix `blk` (`src/kernel/chainparams.cpp`).

Witness v	Program	Purpose
v16	32-byte commitment	Quantum migration output: the ML-DSA-protected home for migrated coins. Subject to demurrage.
v15	32-byte commitment	Reserved EUTXO shape : disabled/frozen in v30.1.1 because it has no quantum ownership authorization.
v14	32-byte commitment	Quantum cold-stake : owner/staker-separated delegation output, subject to inactivity demurrage.

For the authenticated v14 and v16 paths, the on-chain address is a 32-byte commitment and the bulky ML-DSA public key and signature are supplied in the witness at spend time. The reserved v15 commitment does not commit to an ML-DSA owner; this is why v30.1.1 must not fund or spend it.

3.4 Self-test on startup

The node performs an ML-DSA Known-Answer-Test at startup to verify that liboqs is linked correctly and produces standard-conformant signatures before it will participate in consensus. A build that cannot reproduce the ML-DSA KAT refuses to run, guaranteeing that every V4 node validates quantum signatures identically.

4. The Gold Rush Reward Epoch

The Gold Rush is a six-month, deterministic bonus-emission event that launches V4. Its purpose is to **reward existing holders for participating in securing the network at the exact moment the network needs maximum participation**, the transition into the quantum era. It is not an airdrop to passive wallets; it pays holders through the act of staking and mining.

4.1 The whitelist snapshot

At height **5,945,000** (`SHADOW_WHITELIST_HEIGHT`, `src/shadow_schedule.cpp`), the node builds a **deterministic snapshot** of every script holding at least **10,000 BLK aggregate** (`SHADOW_WHITELIST_MIN_BALANCE`, `src/shadow.h`). The snapshot is computed once from the UTXO set at that exact height and is read-only thereafter, so every node derives an identical whitelist.

- Balances are aggregated **per canonical script**. P2PK stake scripts are folded to their P2PKH identity via `CanonicalizeLegacyStakeScript()` so that a holder who staked with raw-pubkey outputs and a holder who used address outputs are treated as one account.
- The snapshot happens **5,000 blocks (~ 3.7 days) before** Gold Rush rewards begin at height 5,950,000, giving the network a clean, pre-announced eligibility set that cannot be gamed after the fact.

Why 10,000 BLK? The threshold defines the set of accounts large enough to be meaningful security participants during the transition. Being whitelisted is necessary to earn Gold Rush *credits*, but it does not by itself pay anything, the holder still has to show up and stake.

4.2 The reward schedule

Each Gold Rush block accrues a base reward, `ShadowBaseReward(height)` (`src/shadow.cpp`), over the window from height 5,950,000 to **6,192,999** (`SHADOW_REWARD_END_HEIGHT`), a span of `SHADOW_GOLD_RUSH_BLOCKS = (180 × 24 × 60 × 60) / 64 = 243,000` blocks.

Phase 1 (heights 5,950,000-6,187,599): a halving curve.

```
reward = (580 BLK) >> (blocks_since_start / 43,200)
```

The reward starts at **580 BLK/block** and halves every `SHADOW_HALVING_INTERVAL_BLOCKS = 43,200` blocks (~ 30 days):

Month	Height range	Reward/block
1	5,950,000-5,993,199	580 BLK
2	5,993,200-6,036,399	290 BLK
3	6,036,400-6,079,599	145 BLK
4	6,079,600-6,122,799	72 BLK
5	6,122,800-6,165,999	36 BLK
6 (part)	6,166,000-6,187,599	18 BLK

Phase 2 (heights 6,187,600-6,192,999): a fixed tail.

```
reward = 463 BLK/block
```

The final ~5,400 blocks pay a flat **463 BLK/block**. The two-phase shape is calibrated so that total Gold Rush accrual lands at the hard cap `SHADOW_MAX_EMISSION = 51,437,700 BLK` (`src/shadow.h`); the pool cannot over-issue beyond this cap regardless of block timing.

4.3 The PoS / PoW split

Each block's base reward is divided **evenly** between two independent reward pools (`src/shadow.cpp`):

```
pos_pool_reward = reward - reward/2    (50%)
pow_pool_reward = reward/2             (50%)
```

- **The Proof-of-Stake half** rewards Blackcoin's native stakers. This is the primary, energy-free path and the one most holders will use: keep your node online, stake, and earn.
- **The Proof-of-Work half** opens a parallel, opt-in participation lane using a deliberately *CPU-friendly, memory-light* Argon2id puzzle (`SHADOW_ARGON2_TIME_COST = 1`, `SHADOW_ARGON2_MEMORY_KIB = 1024` (1 MiB), `SHADOW_ARGON2_LANES = 1`), so ordinary community members, not just ASIC farms, can contribute and claim.

Rewards accumulate into pools and are drawn by **claims**, not paid blindly to whoever found a block, which lets both PoS and PoW participants collect their fair share over the epoch.

4.4 Qualifying and claiming: QCSIGNAL and QCSPROOF

Gold Rush participation is expressed through two on-chain, fee-paying control transactions carried in `OP_RETURN` outputs:

- **QCSIGNAL (Proof-of-Stake side)**. A whitelisted holder who has produced a recent PoS block signals eligibility by broadcasting a QCSIGNAL that references their recent solve. "Recent" is defined by the solver-activity window `SHADOW_SOLVER_ACTIVITY_SECONDS = 14 days` (`SHADOW_SOLVER_ACTIVITY_WINDOW = 18,900 blocks`, `src/shadow.h`). In other words: stake a block, and you have a 14-day window to signal and be credited from the PoS pool. This is the mechanism that ties reward to genuine, ongoing participation rather than to a one-time balance.
- **QCSPROOF (Proof-of-Work side)**. A miner grinds an Argon2id proof (magic `QCSPROOF`, `src/shadow.cpp`) against the target difficulty, a 12-bit base, ASERT- retargeted every 64 blocks within a [10-bit, 18-bit] band, and submits it in a claim transaction that is validated before mempool acceptance. Already-mined blocks through 5,993,199 retain the v30.1.0 first-valid-claim allocation. At the first scheduled halving, height 5,993,200, v30.1.1 begins canonically ranking competing candidates independently of transaction order and evaluates at most 64 Argon2 proofs. The lowest-ranked valid claim receives the fixed PoW pool after each other evaluated valid claimant is reimbursed its actual base fee, capped at 0.01 BLK. At most 0.63 BLK can be reimbursed in one block. Invalid, malformed, and excess claims receive nothing, and all credits still sum exactly to the pre-existing pool.

The wallet automates both. See §9 for the exact RPCs (`sendshadowsignal`, `sendshadowpowclaim`, `setpowmining`, `getgoldrushinfo`).

5. Quantum Migration and the Legacy Lockout

5.1 The problem being solved

Every legacy Blackcoin output is protected by ECDSA and therefore represents standing quantum risk to the whole network, not just to its owner. Leaving that risk open forever would mean the chain's security never actually improves, no matter how good the new cryptography is. V4 therefore treats migration as a **network-wide, time-boxed public project** with a clear deadline.

5.2 The migration path

Any holder moves coins to safety with a single wallet action, `migratetoquantum` (see §9), which sweeps spendable legacy (non-quantum) outputs into a fresh **wallet-backed quantum migration (witness v16)** address. The destination ML-DSA key is generated and **written to the wallet database before any funds move**, and the call refuses to proceed unless that key is confirmed stored, so a migration can never send coins to a key the wallet does not hold.

Critical backup note. ML-DSA keys are *not* derived from the wallet's HD seed. After creating a migration address you **must back up the wallet again**, or a restore from an older backup will not recover the migrated funds. The wallet and this paper both flag this at every step.

During Gold Rush, wallets can create and back up quantum addresses and can dry-run migration planning with an existing wallet-backed address. They cannot fund or spend ordinary v14/v16 outputs. `migratetoquantum` becomes actionable at Migration height 6,193,000 and remains available through height 6,921,999. Gold Rush reward credits are separate authenticated synthetic outputs that remain phase-locked until Gold Rush ends and normal maturity is satisfied.

5.3 The lockout, and why it is a feature

At Final Lockout, **height 6,922,000**, the consensus rule `IsQuantumFinalLockout(nTime, nHeight)` (`src/consensus/params.h`, enforced in `src/validation.cpp`) becomes true, and the script flag `SCRIPT_VERIFY_LEGACY_ECDSA_LOCKOUT` (`src/script/interpreter.cpp`) causes **all legacy ECDSA-signed spends to be permanently rejected** with `legacy-spend-disabled`. Exact authenticated v14 and v16 paths remain enabled. Witness-v15 funding and spending remain rejected with the dedicated EUTXO-disabled rules.

This is deliberately framed as a positive:

- It converts an **unbounded, permanent, network-wide** quantum vulnerability into a **finite, scheduled, individually-avoidable** one. After the deadline, the set of quantum-vulnerable coins can only shrink, never grow.
- The schedule is **generous and loud**: about six months of Gold Rush preparation plus an 18-month Migration phase, with the exact height deadline visible in the wallet, network status RPCs, and this document.
- It is **individually avoidable in one click**. A holder who migrates during the 540-day Migration phase retains spendable control.
- It **strengthens every remaining coin**. Once legacy spends are closed, the entire active supply is quantum-safe, which raises the security floor for everyone who participated.

The lockout is the mechanism that guarantees the migration actually completes, rather than dragging on forever with a permanently vulnerable dormant tail.

6. Demurrage: Liveness as a Public Good

Demurrage is the most misunderstood, and most important, participation mechanic in V4. This section states plainly what it is and what it is not.

It is a liveness rule for quantum holdings. A timely direct-key attestation or an activity spend refreshes the clock. Delegation alone does not. Demurrage applies to eligible quantum outputs left inactive for **more than six months**.

6.1 Exactly which coins are subject

Evaluated per output by `EvaluateDemurrage` (`src/consensus/demurrage.cpp`). The classification, in order:

1. **Demurrage not yet active** -> exempt. Demurrage cannot begin before the migration era (mainnet's height-authoritative schedule ends Migration at block 6,921,999 and activates Final Lockout and demurrage at block 6,922,000).
2. **Explicitly configured exempt scripts** -> exempt. Mainnet currently configures none.
3. **Non-quantum (legacy and everything else) outputs** -> exempt ("`non_quantum`"). Legacy coins are governed by the lockout, not demurrage.
4. **Quantum migration/tiered (v16) and cold-stake (v14) outputs** are subject, but only if inactive beyond the grace period. Eligible direct/tiered v16 keys can use attestations; cold-stake state refreshes through a successful spend/recreation.
5. **Historical v15-shaped outputs** can be classified by the accounting evaluator, but v30.1.1 independently rejects their funding and spending. They have no supported liveness or recreation path, and their metadata is inspection-only.

Thus cold delegation is not a permanent-value shelter. A cold-stake output that never successfully stakes or moves follows the same inactivity curve.

6.2 The inactivity clock and the grace period

For a subject output, the node computes:

```
effective_last_active = max(coin_creation_height,
                           demurrage_activation_height,
                           latest_attestation_height)
inactive_blocks = spend_height - effective_last_active
```

If `inactive_blocks <= DEMURRAGE_GRACE_BLOCKS` (**243,000 blocks = 6 months**), the output is **exempt**, full value, no decay ("`young`" if freshly created/moved, "`attested`" if kept alive by an attestation). Creating, moving, attesting, or receiving the coin all reset the clock.

6.3 The decay curve

Past the grace period, value decays **quadratically** to zero over an 18-month window (`DemurrageRemainingPpm`, `src/consensus/demurrage.cpp`):

```
grace_blocks = 243,000           (6 months)
zero_blocks  = 972,000           (24 months)
decay_window = 729,000           (18 months)
elapsed      = inactive_blocks - grace_blocks
t            = (elapsed / 729,000) x 1,000,000    (parts per million)
remaining    = 1,000,000 - (t² / 1,000,000)       (ppm of value retained)
```

The value burned when such a coin is spent is **nominal - effective**. Consensus recognizes only the effective value as input principal. The transaction fee is then **effective inputs - outputs**, so the burned remainder is not a fee and is never paid to the block producer. A coin stake is governed by the same rule: only effective principal is returned, and its reward remains limited to the ordinary PoS subsidy plus explicit fees.

Decay table (a quantum coin that is never attested, moved, or successfully staked):

Months inactive	Value retained
<= 6 (grace)	100.0%
9	97.2%
12	88.9%
15	75.0%
18	55.6%
21	30.6%
24	0.0% (locked)

Note the shape: the first year barely moves (the curve is quadratic, so early decay is tiny), and the losses only become material deep into the second year of *total* neglect. This is by design, it gives even a careless holder a very long runway, while ensuring that genuinely dead coins eventually stop counting as security weight.

6.4 Staying at 100% is automatic and nearly free

There are three ways to keep a quantum holding at full value, and the wallet handles them for you:

- **Automatic liveness attestation.** A demurrage attestation is a zero-value, fee-only transaction carrying an ML-DSA signature (`senddemurrageattestation`) that resets the clock. An attestation is valid for **6 months** (`DEMURRAGE_ATTEST_VALIDITY_BLOCKS = 243,000`), and the wallet auto-attests at the **3-month** mark (`DEMURRAGE_AUTO_ATTEST_BLOCKS = 121,500`) while it is running and unlocked for quantum/legacy staking. A holder who simply keeps their wallet online never decays.
- **Active cold staking.** Cold-stake (v14) outputs remain subject to demurrage. Each successful coin stake realizes any accrued burn, returns only effective principal plus the ordinary reward, recreates the output, and resets its activity clock. Delegation by itself is not an exemption.
- **Any ordinary use.** Moving, consolidating, or spending resets the clock as a side effect.

The `getdemurragewalletinfo` RPC reports, per output, the nominal amount, the current effective (post-decay) amount, the value that would be burned if spent now, whether an attestation is due, and whether the output is locked. `sweepdemurragedecay` spends outputs that are decaying but still have positive effective value, realizes the burn, and moves the remainder minus the explicit fee to a fresh quantum address. Outputs at zero effective value are permanently locked and are skipped. The GUI surfaces the same information and can send attestations with one click.

In short, timely participation preserves principal. Any decay realized by a spend is destroyed, not redistributed.

7. Quantum Staking: Tiered, Cold, and Pooled

V4 gives quantum coins a rich set of ways to participate and earn rewards. Participation also refreshes activity when it spends and recreates the relevant output.

7.1 Tiered self-staking

A holder can bond quantum coins into a **tiered self-staking** output that encodes a lock schedule directly in the witness program (`QuantumStakeTierProgram`, `src/consensus/quantum_witness.h`):

- **State machine:** `BONDED` -> (initiate unbonding) -> `UNBONDING` until `unlock_height` -> spendable.
- **Unbonding period** is chosen by the holder (in blocks; at 64 s/block, e.g. 40,500 blocks = 30 days) and is stored in the program itself, so the lock is consensus-visible.
- **RPCs:** `fundquantumstakeaddress` (bond), `withdrawquantumstakeaddress` (unbond or withdraw matured funds, the call is state-aware), `getquantumstakeaddressinfo`, `listquantumstakeoutputs`.

Longer, more committed locks express stronger participation and are ranked accordingly in the pool logic below.

7.2 Cold staking: separate the owner key from the staking key

Cold staking (witness v14) splits control into an **owner key** (can withdraw the principal) and a **staker key** (can only mint blocks with the coins), hashed together under the domain tag "*Quantum Quasar Cold Stake v2*" (`src/consensus/quantum_witness.cpp`). This lets a holder keep their principal in cold storage while a hot node, their own or a trusted operator's, stakes on their behalf.

- The delegated principal is always **owner-controlled**; the operator can never move it.
- Cold-stake outputs are subject to the same inactivity schedule. A successful coin stake returns effective principal plus reward and creates a fresh output, but delegation alone does not freeze the clock.
- **RPCs:** `fundquantumcoldstakeaddress`, `withdrawquantumcoldstakeaddress`, `getquantumcoldstakebalance`.

7.3 Operator bonds and the per-pool cap

Operators who stake on behalf of others post a **30-day operator bond** (40,500 blocks, `src/wallet/rpc/staking.cpp`; pool logic in `src/node/quantum_pool.h`), which registers a verified commitment other participants can see. To keep the network decentralized, a **wallet/policy per-pool cap of 20%** (`QUANTUM_POOL_CAP_BPS = 2000`) discourages any single operator from accumulating an outsized share of delegated stake. The cap is a *wallet and delegation policy*, not a consensus rule, it steers new delegations without changing block validity, so it can be tuned by the community without a fork.

- **RPCs:** `fundquantumoperatorbond`, `withdrawquantumoperatorbond`, `getquantumoperatorbondinfo`, `getwalletquantumpoolinfo` (verified value, share in basis points, valid/invalid claim counts, over-cap flag per operator).

7.4 Autonomous redelegation

Delegations do not have to be babysat. The wallet's redelegation engine (`src/wallet/redelegation.h`) automatically moves stake away from operators who stop producing, or away from pools that exceed the cap, subject to careful policy:

- **Trigger:** an operator that produces zero wins for `6 x expected_interval_blocks`.
- **Rate limiting:** a minimum of **1,350 blocks** (~ 1 day) between redelegation attempts, with a matching probation period after a failed attempt, and randomized jitter to avoid a synchronized "thundering herd" of redelegations.
- **Ranking:** it considers the top candidates by liveness improvement and share, and never redelegates into a pool if doing so would breach the cap.

- **RPCs:** `getquantumrelegationinfo` (dry-run/status), `redelegatquantumcoldstake` (manual, with cap enforcement and a dry-run mode).

The net effect: a delegator's coins keep working for them and keep the pool distribution healthy, with no manual intervention.

8. Reserved v15 EUTXO Design and RGB

V4 includes an operational RGB commitment path and retains a reserved EUTXO encoding for inspection and future design work. Those two states must not be treated as equally enabled.

8.1 EUTXO witness v15 is frozen in v30.1.1

The reserved **EUTXO-shaped output** (witness v15, `src/addresstype.h`) commits to a *datum* and *validator script* as `SHA256("Quantum Quasar EUTXO v1" || SHA256(datum) || SHA256(validator))` (`src/script/solver.cpp`). That commitment does not authenticate a quantum owner. A validator-only spend design would therefore permit value movement without the ML-DSA ownership guarantee required by the migration.

v30.1.1 sets `EUTXO_ENABLED` to false. Wallet and raw-transaction construction reject v15 outputs, the `createeutxospend` and `createeutxotransition` RPCs intentionally return a disabled error, and post-Gold-Rush consensus rejects both v15 funding and v15 spends. `decodeeutxospend` and `verifyeutxospend` remain inspection tools only; `importeutxostate` and `listeutxostates` only persist or report metadata and cannot make a v15 output spendable. **Do not send BLK to a witness-v15 address in v30.1.1.**

8.2 RGB, client-side fixed-supply assets

An **RGB commitment** anchors a client-side-validated asset state transition in a zero-cost `OP_RETURN` (`OP_RETURN <RGB1> <32-byte state hash>`, `src/script/solver.cpp`). The heavy asset data lives off-chain and is validated by clients against the on-chain commitment chain, so Blackcoin can carry fixed-supply tokens and assets without bloating the base layer.

- **Tooling:** `createrrgbtransfer`, `acceptrrgbconsignment`, `exportrrgbconsignment`, `importrrgbcontract`, `importrrgbassignment`, `listrrgbassets`, plus raw `decodergbcommitment` / `verifyrrgbconsignment`.

9. Full Wallet and RPC Reference

Every command below is available from `blackcoin-cli` and the Qt debug console. This is the complete Quantum-Quasar-specific surface added on top of the standard Bitcoin/Blackcoin RPC set.

9.1 Chain and schedule

RPC	Purpose
<code>getquantumquasarinfo</code>	V4 phase, activation/Gold-Rush/deadline times, next-block phase
<code>getgoldrushstate</code>	Chain-level Gold Rush (Shadow Network) state
<code>getgoldrushinfo</code>	Gold Rush pools (PoS/PoW amounts), solver counts, wallet qualification
<code>getcirculatingupply</code>	Demurrage-adjusted circulating supply

RPC	Purpose
<code>getquantumpoolinfo</code>	Non-consensus quantum cold-stake pool registry

9.2 Staking, Gold Rush, and mining

RPC	Purpose
<code>getstakinginfo / staking</code>	Read / start-stop legacy PoS staking
<code>reservebalance</code>	Reserve coins from staking/spending
<code>getstakingdonationinfo / setstakingdonation</code>	Read / set staking-reward donation percentage
<code>checkkernel</code>	Test whether an input is a valid PoS kernel now
<code>sendshadowsignal</code>	Broadcast a QQSIGNAL for a recent PoS solve (Gold Rush PoS credit)
<code>sendshadowpowclaim</code>	Grind and submit a QQSPROOF Argon2id PoW claim
<code>setpowmining / getpowmininginfo</code>	Control / inspect the in-process Argon2id miner
<code>optimizeutxoset</code>	Rebuild the UTXO set into equal outputs to maximize PoS yield

9.3 Quantum addresses and migration

RPC	Purpose
<code>getnewquantumaddress / listquantumaddresses</code>	Create / list wallet-backed ML-DSA migration addresses
<code>createquantumkey / createquantummigrationaddress</code>	Generate an ML-DSA keypair / encode a migration commitment
<code>dumpquantumkey / importquantumkey</code>	Export / import an ML-DSA key for a migration address
<code>migratetoquantum</code>	Sweep legacy coins into a quantum migration address
<code>migrategoldrushrewards</code>	Move Gold Rush reward outputs to a fresh quantum address
<code>getmigrationstatus</code>	Migration progress, eligible legacy amount, deadline countdown, advice

9.4 Quantum staking, cold staking, operator bonds

RPC	Purpose
<code>getnewquantumstakeaddress</code>	New tiered self-staking address
<code>fundquantumstakeaddress / withdrawquantumstakeaddress</code>	Bond / unbond-withdraw tiered stake
<code>getquantumstakeaddressinfo / listquantumstakeoutputs</code>	Inspect tiered stake state and outputs
<code>getnewquantumcoldstakingaddress / createcoldstakingaddress</code>	New cold-stake delegation address
<code>fundquantumcoldstakeaddress / withdrawquantumcoldstakeaddress</code>	Fund / unbond-withdraw a delegation
<code>getquantumcoldstakebalance / listquantumcoldstakingdelegations</code>	Inspect cold-stake holdings

RPC	Purpose
<code>importquantumcoldstakingdelegation</code>	Import delegation metadata
<code>fundquantumoperatorbond / withdrawquantumoperatorbond / getquantumoperatorbondinfo</code>	Operate a 30-day operator bond
<code>getwalletquantumpoolinfo</code>	Verified operator registry (value, share bps, claims, over-cap)
<code>getquantumredelegationinfo / redelegatquantumcoldstake</code>	Inspect / execute redelegation (dry-run supported)

9.5 Demurrage

RPC	Purpose
<code>senddemurrageattestation</code>	Send a liveness attestation to reset a quantum output's clock
<code>getdemurragewalletinfo</code>	Per-output decay state, effective value, attestation-due flag
<code>sweepdemurragedecay</code>	Realize decay on still-spendable outputs and move the effective remainder

9.6 EUTXO inspection and RGB

RPC	Purpose
<code>createeutxospend / createeutxotransition</code>	Disabled in v30.1.1; always return the v15 ownership-authorization error
<code>decodeeutxospend / verifyeutxospend</code>	Inspect candidate or known v15-shaped records; never authorize or enable a spend
<code>importheutxostate / listeutxostates</code>	Persist / list inspection-only EUTXO metadata; does not make an output spendable
<code>createregbtransfer / acceptregbconsignment / exportregbconsignment</code>	RGB transfer lifecycle
<code>importregbcontract / importregbassignment / listregbassets</code>	RGB asset management
<code>decodergbcommitment / verifyregbconsignment</code>	Raw RGB inspection

9.7 Wallet maintenance

RPC	Purpose
<code>deladdressbook</code>	Remove a sending-address book entry (CLI parity with the GUI)
<code>burn / burnwallet</code>	Provably destroy specific coins / the whole wallet balance

9.8 The GUI

The Qt wallet adds two dedicated pages:

- **Staking & Mining**, one place for PoS staking, Gold Rush status, the in-process PoW miner, quantum migration, tiered/cold staking, operator bonds, demurrage, RGB, and inspection-only EUTXO metadata. Expensive detail panels load on demand behind a **Refresh details** button so the tab opens instantly even on

very large wallets.

- **Account**, a per-family (Legacy / Quantum / Cold-stake / EUTXO) breakdown of every output, its state (bonded / unbonding / withdrawable), and demurrage exposure, with CSV export. An EUTXO row is a frozen-output warning and inspection surface, not a funding or spend workflow.

The **Unlock Wallet** dialog offers two explicit, mutually-exclusive modes: **For Legacy Staking Only** (mint PoS blocks, no spending or quantum actions) and **Legacy and Quantum Staking** (full unlock, required for any quantum, Gold Rush, migration, or cold-staking transaction, and for the automatic demurrage attestations that keep holdings at 100%).

10. Worked Examples and Community Playbook

Example A, The long-term HODLer (recommended path)

Alice holds 250,000 BLK and wants zero maintenance.

1. **Before V4:** nothing to do. Keep the coins.
2. **During Gold Rush:** she creates and backs up a wallet-backed quantum address. She can dry-run migration planning, but does not fund it yet.
3. **At Migration height 6,193,000 or later:** she runs `migratetoquantum` once. Her coins move to the quantum (v16) address. **She backs up her wallet again** because ML-DSA keys are not in the seed.
4. **Optional:** she runs `fundquantumcoldstakeaddress` to delegate to a cold-staking operator (or her own hot node). Her principal stays owner-controlled and can earn staking rewards. Successful coinstakes refresh the output's activity clock.
5. **Afterward:** if she leaves coins in a direct quantum address, her wallet auto-attests every 3 months while online. If she delegates, she monitors successful staking or moves the output before prolonged inactivity; delegation alone is not an exemption.

Alice prepares during Gold Rush, completes one migration action during Migration, and can optionally add a staking workflow.

Example B, The active staker during Gold Rush

Bob holds 40,000 BLK (above the 10,000 whitelist threshold) and runs a node.

1. His account is captured in the whitelist snapshot at height 5,945,000.
2. During Gold Rush he simply keeps staking. Each time he mints a PoS block, his wallet broadcasts a `sendshadowsignal` within the 14-day window, and he is credited from the PoS pool, on top of his normal staking rewards.
3. If he wants to also work the PoW lane, he enables `setpowmining`; the light 1-MiB Argon2id puzzle lets his ordinary CPU submit `sendshadowpowclaim` proofs for a share of the PoW pool.
4. He migrates during the 18-month Migration phase, before Final height 6,922,000.

Bob is rewarded precisely for the participation he is already doing.

Example C, The forgotten wallet

Carol migrated to a quantum address during Migration and then lost interest, wallet offline.

- For **six months** after demurrage activates: no effect. 100% retained.

- At **12 months** of total inactivity: 88.9% retained, still barely touched.
- If she returns before the terminal **24-month** boundary and submits a valid liveness attestation, the clock resets before decay is realized. If she spends first, the spend burns the accrued difference and moves only the effective remainder.
- If she reaches a full **24 months** without qualifying activity, the output reaches zero effective value and becomes permanently unspendable. No miner or staker receives it, and Carol is not charged a transaction fee to clean up a zero-value output.

The design gives Carol a long recovery window while permanently removing terminally inactive value from effective supply.

Example D, Running a staking pool

Dan wants to stake on behalf of others.

1. He posts a 30-day operator bond (**fundquantumoperatorbond**), registering a verified commitment.
2. Delegators send him cold-stake delegations; their principals remain theirs, and the 20% per-pool cap keeps any one operator (including Dan) from dominating.
3. If Dan's node goes down, delegators' wallets **autonomously redelegate** away from him after the trigger window, so delegators are protected without lifting a finger, and Dan is incentivized to stay reliable.

11. Economic Analysis: Why This Increases Participation

Every mechanic in V4 pushes the same direction, toward a larger, more active, more distributed set of participants.

- **Gold Rush** front-loads a large, deterministic reward (up to 51,437,700 BLK) at the transition, but pays it *only through staking and mining*. It is the strongest possible incentive to bring nodes online exactly when the network most needs breadth of participation. Holding qualifies; participating pays.
- **Demurrage** removes the free-rider equilibrium of classic PoS. In a passive-holding chain, dormant coins retain full influence forever. Under V4, inactive quantum principal loses effective value and realized decay is burned. Active stakers receive only the ordinary subsidy and explicit transaction fees; their benefit is liveness participation and the deflationary reduction of effective supply, not a transfer from inactive holders.
- **The legacy lockout** guarantees the migration completes, so the network's security actually improves rather than carrying a permanent vulnerable tail. A fully-migrated supply is a stronger, more valuable supply for everyone who stayed.
- **Tiered, cold, and pooled staking** lower the barrier to participation: cold delegation lets a holder participate without exposing the owner key on a hot node, and autonomous redelegation keeps the pool landscape healthy and decentralized on its own.

Across all of these mechanics, the V4 equilibrium points every holder toward the same choice. Whether large or small, technical or not, the sensible move is to participate, because participation is where the rewards are and where the activity clock is refreshed, and where the network's future lies. The "do-nothing" strategy carries the weakest returns, yet it remains trivially easy to leave behind.

In summary, Quantum Quasar rewards HODLers in full for helping secure the network, makes that help nearly effortless, and burns realized decay rather than redistributing it.

12. Security Considerations

- **Quantum resistance is opt-in-by-deadline, not instant.** During Gold Rush and Migration (approximately 720 target days), legacy ECDSA coins remain spendable and therefore quantum-exposed. Holders can prepare addresses and backups during Gold Rush, then should migrate during the 540-day Migration phase. At Final Lockout the spendable path becomes quantum-only. This is an explicit, bounded trade-off in favor of a fair, no-surprises migration.
- **ML-DSA keys are outside the HD seed.** The single most important operational rule: **back up the wallet after every new quantum address.** A seed phrase alone does not recover ML-DSA-protected funds. The wallet enforces "key stored before funds move" and warns at each step, but the backup responsibility is the user's.
- **Attestations and consensus determinism.** Demurrage attestations are ML-DSA-signed and bound to the first input's outpoint as a replay anchor, and are validated in-consensus, so every node computes identical effective values and no attestation can be replayed onto a different coin.
- **Witness v15 is not an enabled smart-contract path.** Its datum/validator commitment lacks ML-DSA owner authorization. v30.1.1 disables supported v15 construction in every phase, and consensus rejects v15 outputs and spends from Migration onward. Its decode, verification, and wallet-metadata surfaces are inspection-only.
- **Consensus compatibility is paramount.** Mainnet's whitelist height (5,945,000), Gold Rush boundaries (5,950,000 through 6,192,999), Migration boundaries (6,193,000 through 6,921,999), and Final Lockout height (6,922,000) are consensus rules. The retained timestamp anchors are nominal forecasts, not mainnet phase boundaries. Every node that wishes to remain on the same chain must use identical height values. Operators upgrading or building alternative clients must match them exactly to avoid a chain split.
- **The per-pool cap is policy, not consensus.** It cannot by itself prevent a determined operator from accumulating stake; it only steers the default wallet behavior. Genuine decentralization still depends on delegators choosing diverse operators, which the autonomous redelegation engine actively encourages.

Appendix A: Consensus Constant Reference

Values reflect the v30.1.1 candidate source. The nominal time anchors and durations descend from v30.1.0. v30.1.1 makes the mainnet lifecycle height-authoritative, starts demurrage automatically at Final Lockout, and adds the competing-claim boundary shown below.

Constant	Value	Meaning	Defined in
QUANTUM_QUASAR_MAINNET_V4_TIME	1783835299 (2026-07-12 05:48:19 UTC)	Nominal V4 time anchor; v30.1.1 mainnet lifecycle is height-authoritative	consensus/params.h
QUANTUM_QUASAR_GOLD_RUSH_SECONDS	15,552,000 (180 days)	Gold Rush duration	consensus/params.h
QUANTUM_QUASAR_MIGRATION_SECONDS	46,656,000 (540 days)	Migration window	consensus/params.h
Nominal final-lockout time	1846043299 (2028-07-01 05:48:19 UTC)	Non-authoritative time-schedule reference (V4 + 720 days)	derived

Constant	Value	Meaning	Defined in
SHADOW_WHITELIST_HEIGHT	5,945,000	Balance snapshot height	shadow_schedule.cpp
SHADOW_WHITELIST_MIN_BALANCE	10,000 BLK	Whitelist eligibility threshold	shadow.h
SHADOW_REWARD_START_HEIGHT	5,950,000	Gold Rush rewards begin	shadow_schedule.cpp
MAINNET_SHADOW_COMPETING_CLAIMS_ACTIVATION_HEIGHT	5,993,200	Canonical competing-claim allocation begins	shadow.h
SHADOW_GOLD_RUSH_BLOCKS	243,000 (180 days)	Gold Rush length	shadow_schedule.cpp
SHADOW_REWARD_END_HEIGHT	6,192,999	Gold Rush rewards end	shadow_schedule.cpp
MAINNET_QUANTUM_FINAL_START_HEIGHT	6,922,000	Height-authoritative Final Lockout and automatic demurrage begin	shadow.h
SHADOW_PHASE1_END_HEIGHT	6,187,599	Halving phase ends	shadow_schedule.cpp
SHADOW_HALVING_INTERVAL_BLOCKS	43,200 (~30 days)	Reward-halving period	shadow_schedule.cpp
Phase-1 base reward	580 BLK/block, halving	Gold Rush emission (halving)	shadow.cpp
Phase-2 base reward	463 BLK/block, flat	Gold Rush emission (tail)	shadow.cpp
SHADOW_MAX_EMISSION	51,437,700 BLK	Gold Rush issuance cap	shadow.h
PoS / PoW split	50% / 50%	Per-block reward pool split	shadow.cpp
SHADOW_SOLVER_ACTIVITY_WINDOW	18,900 blocks (14 days)	Recent-PoS-solve window for signalling	shadow.h
Argon2id (time, mem, lanes)	1, 1024 KiB, 1	PoW puzzle parameters	shadow.cpp
DEMURRAGE_GRACE_BLOCKS	243,000 (6 months)	No-decay grace period	consensus/demurrage.h
DEMURRAGE_ZERO_BLOCKS	972,000 (24 months)	Full-decay point	consensus/demurrage.h
Decay window	729,000 (18 months)	Quadratic decay span	consensus/demurrage.cpp
DEMURRAGE_ATTEST_VALIDITY_BLOCKS	243,000 (6 months)	Attestation validity	consensus/demurrage.cpp
DEMURRAGE_AUTO_ATTEST_BLOCKS	121,500 (3 months)	Wallet auto-attest cadence	consensus/demurrage.cpp
DEMURRAGE_BLOCKS_PER_MONTH	40,500	Block/month conversion	consensus/demurrage.h
ML-DSA public key	1,312 bytes	Quantum public key size	crypto/mldsa.h
ML-DSA signature	2,420 bytes	Quantum signature size	crypto/mldsa.h
Quantum migration witness	v16, 32-byte program	Migrated-coin output	consensus/quantum_witness.h
EUTXO witness	v15, 32-byte program	Reserved shape; funding and spending disabled in v30.1.1	addresstype.h

Constant	Value	Meaning	Defined in
Cold-stake witness	v14, 32-byte program	Delegation output (subject to inactivity demurrage)	consensus/quantum_witness.h
QUANTUM_POOL_CAP_BPS	2000 (20%)	Per-pool delegation cap (policy)	node/quantum_pool.h
Operator bond period	40,500 blocks (30 days)	Verified operator commitment	wallet/rpc/staking.cpp
Block time	64 seconds	Mainnet target spacing	kernel/chainparams.cpp
Bech32 HRP	blk	Mainnet address prefix	kernel/chainparams.cpp

Appendix B: Glossary

- **ML-DSA-44**, Module-Lattice Digital Signature Algorithm (FIPS 204), the post-quantum signature scheme used for all quantum spends and attestations.
- **Gold Rush**, the 180-day bonus-emission epoch that launches V4, paid to stakers and miners.
- **Whitelist**, the deterministic set of scripts holding $\geq 10,000$ BLK at height 5,945,000, eligible to earn Gold Rush PoS credits.
- **QQSIGNAL / QQSPROOF**, the OP_RETURN control transactions that claim Gold Rush PoS and PoW rewards respectively.
- **Migration**, moving legacy ECDSA coins into quantum (v16) outputs via `migratetoquantum`.
- **Final Lockout**, mainnet height 6,922,000, after which legacy ECDSA spends are permanently rejected and demurrage begins automatically.
- **Demurrage**, the liveness mechanism by which inactive quantum outputs slowly lose effective value. Realized decay is burned, never paid to a miner or staker. Timely attestation or a spend/recreation refreshes activity; cold delegation alone does not.
- **Attestation**, a zero-value ML-DSA-signed transaction that resets a quantum output's demurrage clock; auto-sent by the wallet.
- **Cold staking**, owner/staker key separation (witness v14) letting a holder delegate staking while retaining principal control; subject to the inactivity schedule.
- **Tiered staking**, self-staking with a consensus-visible bonding/unbonding lock schedule.
- **Operator bond**, a 30-day verified commitment posted by a staking-pool operator.
- **EUTXO**, a reserved witness-v15 datum/validator commitment shape. v30.1.1 freezes its funding and spending pending a quantum-authenticated ownership design.
- **RGB**, client-side-validated, fixed-supply asset commitments anchored on-chain.

This document describes the Blackcoin Quantum Quasar (Protocol V4) v30.1.1 candidate. All consensus boundaries are drawn from that candidate source. Blackcoin is free/open-source software under the MIT license.